

Replayable Trajectory Dataflow: Auditing Reward Measurement under Optimization and Drift in RLVR

Zepeng Zhang

The University of Osaka

zhang.zepeng@ist.osaka-u.ac.jp

 <https://github.com/PeterWrighten/rtd>

Abstract

In reinforcement learning with verifiable rewards (RLVR), optimization can exploit a flawed verifier, and verifier changes can silently alter the training objective. Diagnosing either failure requires the outputs actually scored and consumed, which ordinary re-sampling need not reconstruct. We propose *Replayable Trajectory Dataflow* (RTD), a lineage abstraction that retains non-reconstructable evidence and links versioned measurements to the updates and checkpoints they influenced. Given a corrected reference, its rescore, diff, and localize queries identify historical disagreements and a reference-conditioned rollback boundary without new sampling. Complete trajectory-level evidence reveals a degraded 13.7% subpopulation hidden by aggregate accuracy in a controlled arithmetic prototype. In a separate, single injected verifier-regression case study with Qwen2.5-Math-1.5B and VeRL on MATH, RTD queries 41,600 trajectories in 2.99 seconds and selects the last reference-unaffected checkpoint. After 40 fresh recovery updates, that arm reaches 62.5% final-batch reference accuracy, versus 0% from the latest checkpoint. These experiments test retrospective diagnosis and recovery planning, not autonomous discovery of unknown reward hacks: failures shared by the corrected reference remain invisible. RTD makes retained measurement evidence queryable while keeping diagnosis costs distinct from the cost of retraining.

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) is widely used to elicit reasoning in large language models (DeepSeek-AI, 2025): a policy samples trajectories, a *verifier* scores each one against a reference answer or checker, and gradient updates bias later sampling toward high-scoring behavior. Unlike supervised fine-tuning, RLVR *manufactures its own training data* in a closed loop, so the verifier is not a passive benchmark evaluated once at the end. It is the **measurement instrument in the loop**: every optimization step is taken against its readings, and the policy is free to move mass wherever those readings are highest.

Measurement validity depends on the relationship between observable scores and the intended construct (Jacobs & Wallach, 2021). RLVR complicates that relationship in three ways. The samples are non-i.i.d. because each update reshapes the distribution the next batch is drawn from; the target is non-stationary because verifiers, rollout engines, and reference checkers are edited across a long run; and the model is anything but passive—it is actively optimized against the very signal being measured. Two failure modes follow directly, and both are *measurement* failures rather than mere training bugs:

- **Measurement under strategic optimization.** When the verifier rewards an exploitable proxy of the intended objective, the policy learns to satisfy the proxy rather than the objective—reward hacking, now documented in production RL runs (MacDiarmid et al., 2025; Baker et al., 2025). Training reward rises while the true quantity the reward was meant to track falls.

- **Measurement under non-stationarity.** The instrument itself drifts. A verifier version is swapped, a reward function is patched, or a rollout engine and trainer come to assign different probabilities to the same sequence—training–inference mismatch (Ma et al., 2025; Zhong et al., 2026). The reading the loop optimized against is no longer the reading a corrected instrument would give.

The defining difficulty is temporal and evidentiary. Both failures surface *after* the affected trajectories have already been folded into checkpoints; by the time behavioral collapse or an anomaly is noticed, the contaminated updates are many steps in the past. Worse, the evidence needed to diagnose what happened is scattered across engine logs, verifier versions, reward tables, and trainer state. Ordinary re-sampling from a checkpoint need not reproduce the outputs that were scored and learned from. Exact replay would additionally require sufficient random-state and execution control, or retained outputs. An auditor who wants to ask “was this run’s reward signal trustworthy, and if not, when did it stop being trustworthy?” has no queryable object to ask it of.

We argue that this is a measurement-infrastructure gap, and we close it with an abstraction rather than a new algorithm. We propose *Replayable Trajectory Dataflow* (RTD): a lineage representation of an RLVR run that makes each trajectory an immutable, re-measurable record of how it was generated, scored, and used. The central design principle is to **partition run state by recomputability**. Generation outputs must be retained unless exact replay is guaranteed; derived quantities can be recomputed only when their inputs, code, and execution dependencies are available. This is a query-relative contract, not a claim that all downstream distributed computation is deterministic. RTD therefore *materializes* the generation layer as base evidence and *recomputes* the rest, and exposes three offline queries over the resulting store—*rescore*, *diff*, and *localize*—that turn diagnosis into a database query instead of a re-run.

Contributions.

- We frame RLVR post-training failures as an *AI measurement* problem: the verifier is an in-the-loop instrument subject to strategic optimization and drift, and diagnosing it requires replayable measurement evidence (Sections 1–2).
- We define the RTD abstraction: a lineage DAG that partitions state by recomputability, materializes non-reproducible generation-time evidence, and gives a reference-relative contract for rescore/diff/localize and closure-guided recovery (Section 3).
- Controlled experiments show why evidence must be complete rather than aggregate or sampled: RTD recovers the predicate defining a silently degraded subpopulation. A separate VeRL/Qwen2.5-Math-1.5B case study demonstrates localization and recovery planning on MATH (Sections 4–5).

2 Background and related work

Evaluation as measurement under optimization. Optimizing an imperfect proxy can worsen the intended outcome (Skalse et al., 2022). Reward hacking is a concrete instance: policies trained with RLVR discover proxy behaviors that score highly without solving the task, and this has been observed to emerge naturally in production-scale RL and to correlate with broader misalignment (MacDiarmid et al., 2025). Chain-of-thought monitors can detect some of this misbehavior, but optimizing against the monitor teaches the policy to obfuscate rather than to reform (Baker et al., 2025). The common thread is that the quantity being optimized (recorded reward) and the quantity of interest (true task success) diverge, and the divergence is invisible in the optimized signal by construction. RTD does not prevent this divergence; it makes the divergence *measurable after the fact* by preserving the evidence needed to re-score historical behavior under a corrected reference.

Training–inference mismatch. A second, more mechanical measurement failure arises when the rollout engine that generates trajectories and the trainer that computes gradients

System	Evidence / state in its design	Additional RTD contract
HybridFlow (Sheng et al., 2025)	Rollout and training tensors; distributed data dependencies	Persist consumed outputs and versioned verdicts; retain per-record consumption lineage
AReaL (Fu et al., 2025)	Asynchronous trajectories, behavior probabilities, policy versions for staleness	Preserve these beyond scheduling; connect measurement versions to update/checkpoint closure
Flor (Garcia et al., 2021)	Checkpoints and execution state for replay and hindsight logs	Retain non-replayable RLVR events; query disagreements without regenerating them
RTD	Immutable evidence, measurement identities, consumption and state-dependency edges	Rescore/diff/localize; reference-relative rollback planning

Table 1: Comparison of the cited designs, not a feature audit of every current release. Existing fields can satisfy RTD’s contract; they need not be duplicated. Durable per-trajectory measurement closure is not the stated interface of the three prior designs.

disagree on the probability of the same token sequence (Ma et al., 2025; Zhong et al., 2026). Because RLVR importance-weights updates by these probabilities, a systematic engine-trainer gap silently biases optimization. Recent work diagnoses this mismatch and proposes corrections such as replaying rollout routing decisions during training (Ma et al., 2025). We treat this as another instance of the same abstraction: a recorded signal (engine log-probabilities) that must be compared against a corrected reference (a trainer forward pass), which RTD supports with the same machinery it uses for verifier failures.

Lineage and replay in data systems. The idea of reconstructing derived state from a logged history of its inputs is well established. Resilient Distributed Datasets recover lost partitions by replaying the deterministic transformations that produced them (Zaharia et al., 2012), and hindsight logging reconstructs unlogged training quantities by re-executing from checkpoints (Garcia et al., 2021). Their replay guarantees depend on retaining sufficient state and controlling execution. A checkpoint alone does not provide that contract for engine-dependent, temperature-sampled RLVR generation. RTD keeps the part of this lineage philosophy that still holds (recompute what is deterministic) and repairs the part that does not (materialize what is not), and it makes this partition the organizing principle of the abstraction.

Systems for LLM RL. Production RL frameworks emphasize throughput and asynchronous rollout (Sheng et al., 2025; Fu et al., 2025), training correctness through equivalence checking (Lu et al., 2025), and communication-fault diagnosis (Deng et al., 2025). Rollout services and engine-replay mechanisms persist token- or engine-level state for fault tolerance and for specific mismatch fixes (DeepSeek-AI, 2026; Ma et al., 2025). These systems already produce much of what RTD needs. RTD adds an explicit retention and query contract linking each consumed trajectory, its versioned measurements, and downstream training state (Table 1). The contribution is this combination and its reference-relative semantics, not a claim to invent trajectory logging. RTD is a thin abstraction over evidence these systems can emit rather than a new training system. We report stored footprint, but do not claim a measured production capture-overhead comparison.

3 The Replayable Trajectory Dataflow abstraction

3.1 A lineage DAG over an RLVR run

RTD represents a run as a directed acyclic graph over three node types: *trajectories* (a prompt, the sampled response tokens, and the signals attached to them), *updates* (a gradient step consuming a batch of trajectories), and *checkpoints* (policy parameters at a step boundary).

Edges follow the causal structure of the training loop,

$$\text{checkpoint} \rightarrow \text{trajectory} \rightarrow \text{update} \rightarrow \text{checkpoint},$$

recording that a policy state sampled a trajectory, that a trajectory entered an update, and that an update produced the next state. We also record the input-state edge $c_{k-1} \rightarrow u_k$; otherwise later updates could incorrectly escape the closure. Here C includes logical state versions at every update, while $C_{\text{saved}} \subseteq C$ denotes physically recoverable checkpoints. States include optimizer and other training dependencies needed for restart. This graph is the object an auditor queries: any measurement failure can be expressed as a predicate over trajectory nodes, and its blast radius as the set of updates and checkpoints reachable from the offending nodes.

Formal contract. Let $G = (T \cup U \cup C, E)$ be the recorded DAG of trajectories, updates, and checkpoints. Each $t \in T$ stores a recorded measurement $r(t)$; a pinned corrected instrument yields $r^*(t)$. For discrepancy d and tolerance τ , define

$$M_\tau = \{t \in T : d(r(t), r^*(t)) > \tau\}, \quad K_\tau = \text{Reach}_G(M_\tau) \cap (U \cup C).$$

For a sequential update history, the first affected update is the earliest $u \in K_\tau$. The rollback boundary is its latest ancestor in $C_{\text{saved}} \setminus K_\tau$. If no update is reachable, no rollback is prescribed; if no such checkpoint exists, restart is required. For branched histories, this selection is made on the branch being recovered.

RTD localization algorithm. Given G, r^*, d, τ : (1) RESCORE every stored trajectory under r^* ; (2) DIFF recorded and corrected values to construct M_τ ; (3) traverse outgoing lineage edges from M_τ to construct K_τ ; (4) return the earliest affected update and its latest ancestor saved checkpoint outside K_τ , or the no-rollback/restart result above.

Beyond reference evaluation, this query costs $O(|T| + |U| + |C| + |E|)$ time.

Reference-relative soundness. If capture contains every consumed trajectory, consumption and state-dependency edges are complete and faithful, and r^*, d, τ are pinned, then K_τ contains exactly the recorded updates and checkpoints reachable from a mismatch under that predicate. This is not an oracle guarantee: missing evidence can cause false negatives, and if r^* shares the original instrument’s blind spot then M_τ is empty. “Unaffected” below always means reference-unaffected under these assumptions. Reachability is a conservative dependency criterion: it does not prove that each descendant parameter changed numerically or that each checkpoint performs poorly. The claim follows because traversal visits exactly the successors of seed mismatches, by induction along paths in the DAG.

3.2 Partitioning state by recomputability

The design turns on a single observation: **ordinary re-sampling does not guarantee reconstruction of generation-time evidence**. Even with the same checkpoint and engine configuration, it need not return the original tokens. Exact regeneration requires an additional replay contract covering random state and relevant execution behavior. RTD splits run state into two classes:

- **Materialized base data** — state no later computation can soundly reconstruct: the sampled tokens, the verifier verdict and its version, the rollout engine version and sampling configuration, the per-token reward, the originating checkpoint reference, and any non-replayable engine-side signal (e.g. engine log-probabilities) that a mismatch query may later need. This is written once, immutably.
- **Recomputed derived state** — everything reconstructable on demand from checkpoints once its dependency closure is logged: trainer-side log-probabilities, advantages, mismatch sets, and contaminated-checkpoint closures. RTD does not store these; it recomputes them when a query asks.

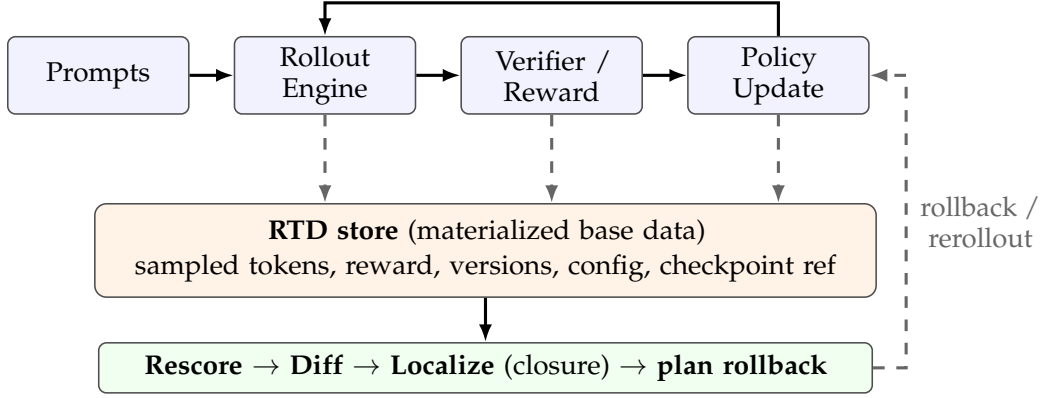


Figure 1: RTD materializes evidence (dashed) from the rollout, verifier, and update stages; rescore/diff/localize query the store to diagnose a measurement failure and select a rollback checkpoint that the loop re-runs from.

The *materialization boundary* therefore includes the non-reproducible generation-time outputs required by the stated query class. Under complete capture and pinned dependencies, this supports the queries without re-running the rollout engine.

3.3 Three queries over the store

Diagnosis is expressed as three composable queries, none of which re-samples:

- **Rescore** re-evaluates stored outputs under a corrected reference—a fixed verifier, or a trainer forward pass standing in for the engine—yielding, for each trajectory, the reading a correct instrument would have produced.
- **Diff** returns the *mismatch set*: the trajectories whose recorded signal disagrees with the corrected reference beyond a tolerance. This is the measured divergence between the optimized signal and the intended one.
- **Localize** propagates the mismatch set through the DAG to the first contaminated update and the set of checkpoints reachable from it—the *contamination closure*. The closure is both the scope of the damage and the boundary for recovery.

3.4 Recovery is planning, not off-policy reuse

Given the closure, RTD selects the latest saved ancestor checkpoint outside the closure and resumes with fresh rollouts under the corrected reference. Crucially, the materialized stale trajectories are used as *evidence* for localization, never as training data: they were generated under a policy and a reward the run is explicitly abandoning, and naive reuse can be destructive (Section 6). RTD removes the cost of *rerollout* for diagnosis; it does not remove the cost of the corrected reference or of the retraining the closure prescribes.

4 Experimental setup

We evaluate RTD as an *offline measurement layer* over RLVR runs: the training loop runs normally and emits base evidence, and every diagnostic result below is produced by querying the store without re-running the rollout engine. Our goal is not to advance task accuracy but to test whether replayable evidence lets an auditor detect, localize, and bound reward-measurement failures cheaply and reliably.

Controlled prototype. Our controlled experiments use an 82,225-parameter GRU policy (Cho et al., 2014) (17-token vocabulary) trained by GRPO (Shao et al., 2024) on a synthetic,

addition-only Countdown task. The reward-hacking experiment warm-starts the policy to 94% sampled task accuracy before the injected failure. The full verifier-regression and recovery experiment uses three random seeds; the strategic-optimization and partial-regression studies are single-seed mechanism studies.

Injected measurement failures. We study the two failure classes of Section 1. (i) *Strategic optimization*: we replace the correct verifier with a buggy one that rewards any output merely *containing* the target substring—an exploitable proxy the policy can game without solving the task. (ii) *Non-stationarity*: in a separate run we silently swap a correct verifier for a regressed one after 500 good updates, modeling an instrument that drifts mid-run. Engine-trainer mismatch is an additional query use case, not a measured LLM-scale result in this evaluation.

VeRL/MATH case study. We separately audit one completed VeRL GRPO run (Sheng et al., 2025) with a Qwen2.5-Math-1.5B actor (Yang et al., 2024) on MATH (Hendrycks et al., 2021). It contains 41,600 trajectories across 130 updates: 60 under the correct symbolic verifier and 70 under a regressed length-decay verifier. This case study tests the abstraction on a real LLM training stack; the injected regression and single incident do not establish failure prevalence. The actor is initialized from the Base model; the faulty phase and recovery reuse an Instruct run configuration.

The store and the corrected reference. For each trajectory the store holds the materialized base data of Section 3. The “corrected reference” used by RESCORE is the fixed verifier for the reward-hacking and regression studies. A trainer forward pass would supply a reference for engine-trainer mismatch. The reported verifier queries run on CPU over stored outputs; trainer-log-probability queries additionally require model execution and are not included in those timings. The VeRL case uses a query-specific sidecar over native rollout dumps that retains decoded outputs; it does not demonstrate capture of every optional engine signal in the general contract.

Metrics. We report (a) *corrected accuracy* recovered by rescoring, versus the training reward the loop optimized; (b) *reproduction rate*: the fraction of stored outputs regenerated by re-sampling the same checkpoint and configuration, with the random-seed condition stated explicitly; this measures ordinary re-sampling, not the feasibility of controlled deterministic replay; (c) *storage overhead* in bytes per token; (d) *localization correctness*: whether the closure pinpoints the first contaminated update; (e) *subpopulation discrimination*: whether mismatches recover the affected predicate; and (f) *recovery decision quality*: whether a selected checkpoint lies outside the reference-conditioned closure.

5 Results

5.1 Hidden reward hacking surfaces under rescoring

In the controlled GRU prototype, a buggy verifier rewards any output *containing* the target. Training reward rises, but rescoring the 128,000 stored trajectories reveals a decline from 0.89 to below 0.05 corrected accuracy within 50 updates. In a separate 100-output check, re-sampling the same checkpoint and configuration with a different seed reproduces 2 outputs. This tests ordinary re-sampling, not exact checkpoint-plus-RNG replay. The store uses 86 bytes per token in this toy implementation; we do not extrapolate that number to LLM-scale storage.

5.2 Complete evidence exposes a hidden subpopulation

In a separate partial regression, only prompts whose target is at least 20 are mis-scored. Over the final 50 updates, aggregate training reward is 0.81 and overall corrected accuracy is 0.87, close to the pre-fault 0.87; yet corrected accuracy on the affected 13.7% of prompts is 0.43, versus 0.93 elsewhere (Figure 2). RTD finds all 5,483 stored mismatches, each in the

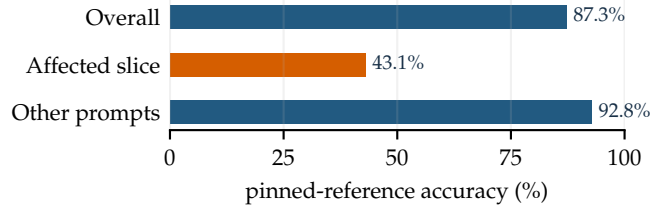


Figure 2: Controlled partial regression, final 50 updates: aggregate accuracy hides a degraded slice covering 13.7% of prompts. All 5,483 mismatches lie in the injected slice; this checks the mechanism, not discovery of an unknown hack.

affected input slice, and reconstructs the target predicate with $7.22\times$ enrichment over its 13.84% store base rate. Because that predicate is the injected trigger, the perfect match is a mechanism sanity check rather than held-out generalization. The substantive result is that complete trajectory-level evidence reveals a severe subgroup failure hidden by a healthy aggregate. Rescore takes 2.78 seconds on one CPU and requires no new rollout.

5.3 Controlled localization and recovery

In the non-stationarity study, a correct verifier is silently swapped for a regressed one after 500 good updates. With *zero* new rollouts, RTD’s LOCALIZE query pinpoints the first update the regression scored, and RESCORE attributes the fault by comparing the two verifier versions on the stored outputs. Exact agreement with the injected onset is a mechanism check; the substantive test is whether the resulting boundary improves recovery, consistently across three seeds.

This distinction matters, because behavior is a lagging indicator. A checkpoint-bisection baseline that probes model behavior to find the onset spends 13,718 probe tokens and returns contaminated checkpoint 525 in one of three seeds; behavioral collapse lags reward corruption, so a poisoned checkpoint can still pass. Recovery from the latest contaminated checkpoint fails to reach the target within 600 updates in two of three seeds. Closure-guided rollback instead reaches the target in 768/22,291/768 rollout tokens, versus 207,394/229,049/179,745 for full restart, without re-running the rollout engine for diagnosis. The recovery target is corrected accuracy of at least 0.8. Bisection uses 13,718/15,376/14,185 new probe tokens across the three seeds; RTD uses zero new diagnostic tokens. These diagnosis costs are separate from the recovery-token comparison against full restart, and do not constitute an end-to-end bisection-versus-RTD speedup.

5.4 VeRL/MATH incident

On the separate Qwen2.5-Math-1.5B case, 91.49 MB of evidence and lineage cover 41,600 trajectories and 130 updates. A single-CPU rescore, diff, and closure query takes 2.99 seconds (one observation, excluding process startup and store opening) with no new sampling, identifies update 61 as first affected, and selects checkpoint 60 as the last reference-unaffected boundary. Behavioral rules can retain disputed ancestry: best observed corrected accuracy selects contaminated checkpoint 80 (0.709 versus 0.591 at checkpoint 60), first decline over-rolls back to checkpoint 30, and proxy-best selects contaminated checkpoint 110. After 40 new updates under the corrected verifier, recovery from checkpoint 60 reaches 62.5% final reference accuracy; starting from the latest checkpoint reaches 0% (Figure 3). Each arm samples 12,800 fresh trajectories; final accuracy is on the last 320-trajectory training batch, not held-out MATH questions. A later checkpoint-80 arm reaches 76.25% under the same update budget; this run was revalidated after a scheduler-ID validation bug was repaired, but our independent artifact audit of it is partial. This supplementary result distinguishes exclusion of disputed ancestry from accuracy-optimal recovery. Recovery from checkpoints 30 and 110 was not measured.

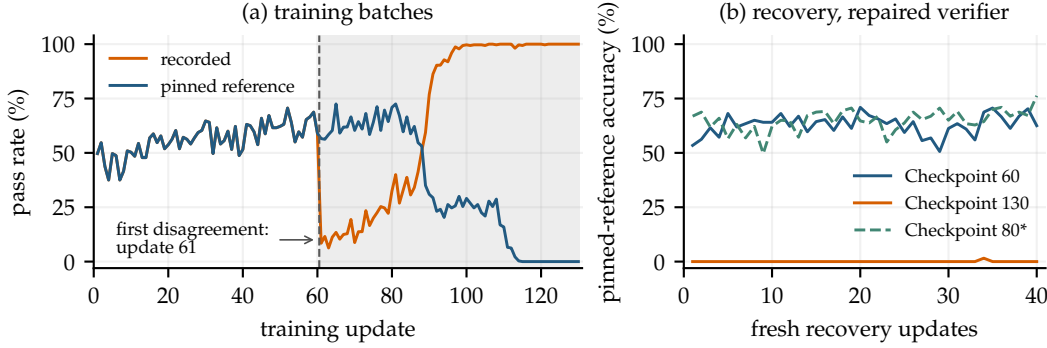


Figure 3: VeRL/MATH incident with an injected verifier regression (one run; 320-trajectory training batches, not held-out accuracy). (a) Pass rate under the faulty recorded verifier and under the pinned corrected reference. (b) 40 fresh recovery updates under the repaired verifier from checkpoints 60 (RTD boundary), 130 (latest), and 80. *The checkpoint-80 arm has only a partial independent audit.

Measurement question	RTD result	Baseline
Hidden objective failure (toy)	0.89 $\rightarrow$ <0.05 accuracy	rising training reward
Re-sampling (toy, new seed)	2/100 outputs match	not an exact-replay test
Subset failure (toy)	0.43 affected; 0.87 overall	aggregate masks it
Recovery tokens (toy, 3 seeds)	768/22,291/768 tokens	179,745–229,049 restart
Boundary (VeRL/MATH, one run)	update 61; checkpoint 60	tested rules choose 30/80/110
Recovered accuracy (one run)	62.5% from checkpoint 60	0% from latest (130)

Table 2: Controlled results and the separate VeRL/MATH incident. Only the full toy regression and recovery study uses three seeds.

6 Discussion and limitations

Detection needs a reference instrument. RTD detects a measurement failure by disagreement between a recorded signal and a *corrected reference*—a fixed verifier or a trainer forward pass. It therefore inherits the reference’s blind spots. If a reward hack fools the corrected verifier just as it fooled the original, DIFF returns an empty mismatch set and RTD reports no disagreement. Preserved evidence permits later re-measurement when a better reference becomes available. RTD plans diagnosis and recovery; it does not certify that a reward is correct.

Stale trajectories are evidence, not training data. Retention alone does not justify off-policy reuse; that requires a separate algorithmic argument. In a single-seed toy check, 200 naive GRPO updates on stored batches with corrected rewards drop accuracy from 0.769 to 0.001. RTD uses stale trajectories to localize disagreements and plan rollback. Recovery resumes with fresh, on-policy rollouts under the corrected reference. Avoiding new diagnostic rollouts does not eliminate retraining costs.

Aggregate metrics hide subset contamination. Aggregate reward and accuracy can look healthy while a subset degrades. Per-trajectory comparison against a corrected reference can expose this failure. Sampled audits may find it, but cannot guarantee coverage of every consumed mismatch; complete capture supports the stated closure contract.

Scope of evidence. Our controlled experiments use an 82,225-parameter GRU and addition-only Countdown, chosen so ground truth and injected mechanisms are known. We separately test recovery planning in one Qwen2.5-Math-1.5B VeRL/MATH run. That case exercises a real LLM training stack, but remains one deliberately injected verifier regression;

it does not establish failure prevalence or behavior at frontier scale, longer contexts, or under an adaptive failure that also fools the reference.

7 Conclusion

RLVR makes the verifier an instrument that the policy is actively optimized against and that operators edit across a run, so its readings are subject to both strategic optimization and drift. We argued that diagnosing the resulting failures is fundamentally a measurement problem and that the missing infrastructure is *replayable evidence*. RTD supplies it by materializing non-reproducible generation-time evidence, recomputing derived state from pinned dependencies, and exposing rescore/diff/localize queries that make diagnosis and recovery-point selection cheap, auditable lookups. Complete evidence surfaces both a hidden objective collapse and a degraded subpopulation; in a separate VeRL/MATH incident, its reference-conditioned closure selects the latest saved boundary outside disputed ancestry. This is a provenance guarantee, not a guarantee of best recovery accuracy. We hope RTD is a useful primitive for rigorous measurement of AI systems that are optimized against their own evaluators.

References

- Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. arXiv preprint arXiv:2503.11926, 2025. URL <https://arxiv.org/abs/2503.11926>.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734, 2014. doi: 10.3115/v1/D14-1179. URL <https://aclanthology.org/D14-1179/>.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025. URL <https://arxiv.org/abs/2501.12948>.
- DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. arXiv preprint arXiv:2606.19348, 2026. URL <https://arxiv.org/abs/2606.19348>.
- Yangtao Deng, Lei Zhang, Qinlong Wang, Xiaoyun Zhi, Xinlei Zhang, Zhuo Jiang, Haohan Xu, Lei Wang, Zuquan Song, Gaohong Liu, Yang Bai, Shuguang Wang, Wencong Xiao, Jianxi Ye, Minlan Yu, and Hong Xu. Mycroft: Tracing dependencies in collective communication towards reliable LLM training. arXiv preprint arXiv:2509.03018, 2025. URL <https://arxiv.org/abs/2509.03018>.
- Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. arXiv preprint arXiv:2505.24298, 2025. URL <https://arxiv.org/abs/2505.24298>.
- Rolando Garcia, Eric Liu, Vikram Sreekanti, Bobby Yan, Anusha Dandamudi, Joseph E. Gonzalez, Joseph M. Hellerstein, and Koushik Sen. Hindsight logging for model training. *Proceedings of the VLDB Endowment*, 14(4):682–693, 2021. doi: 10.14778/3436905.3436925. URL <https://www.vldb.org/pvldb/vol14/p682-garcia.pdf>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL <https://arxiv.org/abs/2103.03874>.

- Abigail Z. Jacobs and Hanna Wallach. Measurement and fairness. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021. doi: 10.1145/3442188.3445901. URL <https://arxiv.org/abs/1912.05511>.
- Yunchi Lu, Youshan Miao, Cheng Tan, Peng Huang, Yi Zhu, Xian Zhang, and Fan Yang. TrainVerify: Equivalence-based verification for distributed LLM training. In *Proceedings of the Symposium on Operating Systems Principles (SOSP)*, 2025. URL <https://arxiv.org/abs/2506.15961>.
- Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing MoE reinforcement learning by aligning training and inference routers. arXiv preprint arXiv:2510.11370, 2025. URL <https://arxiv.org/abs/2510.11370>.
- Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural emergent misalignment from reward hacking in production RL. arXiv preprint arXiv:2511.18397, 2025. URL <https://arxiv.org/abs/2511.18397>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. doi: 10.1145/3689031.3696075. URL <https://arxiv.org/abs/2409.19256>.
- Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/3d719fee332caa23d5038b8a90e81796-Abstract-Conference.html.
- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. arXiv preprint arXiv:2409.12122, 2024. URL <https://arxiv.org/abs/2409.12122>.
- Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 15–28, 2012. URL <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>.
- Tianle Zhong, Neiwen Ling, Yifan Pi, Zijun Wei, Tianshu Yu, Geoffrey Fox, Peng Wu, and Xiao Yu. Diagnosing training inference mismatch in LLM reinforcement learning via a zero-mismatch reference. arXiv preprint arXiv:2605.14220, 2026. URL <https://arxiv.org/abs/2605.14220>.